
BugHunter: An Automated Game Test Framework with MARL-Based Data Collection and FDM-Based Bug Detection

Jung In Kim¹, Jungmin Lee¹, Jaehoon Kim¹,
Jongkook Heo¹, Jinyong Jeong¹, and Seoung Bum Kim¹

¹School of Industrial and Management Engineering, Korea University



Introduction

What is Game Quality Assurance (QA)?

❖ The process of checking the **overall quality of a game**, before its commercial release

1. Whether any bugs occur during gameplay
 2. Whether the different factions are properly balanced
 3. Whether any issues occur within the game map
- ...

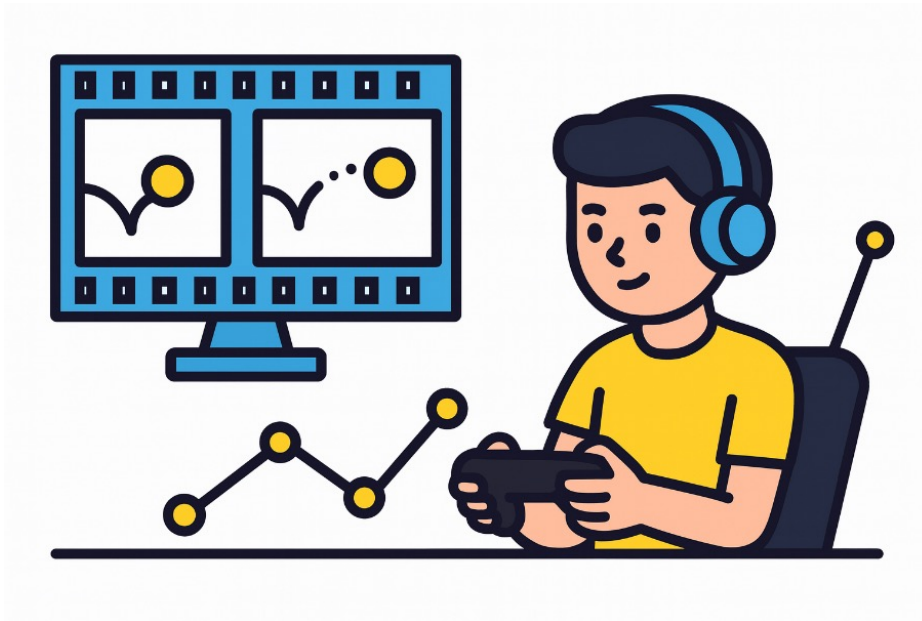


newly released games

Introduction

What are the limitations of Game Quality Assurance (QA)?

- ❖ Traditionally, game QA has been done by human testers
- ❖ Checking the gameplay to see **whether anything goes wrong**



Data Collection Phase

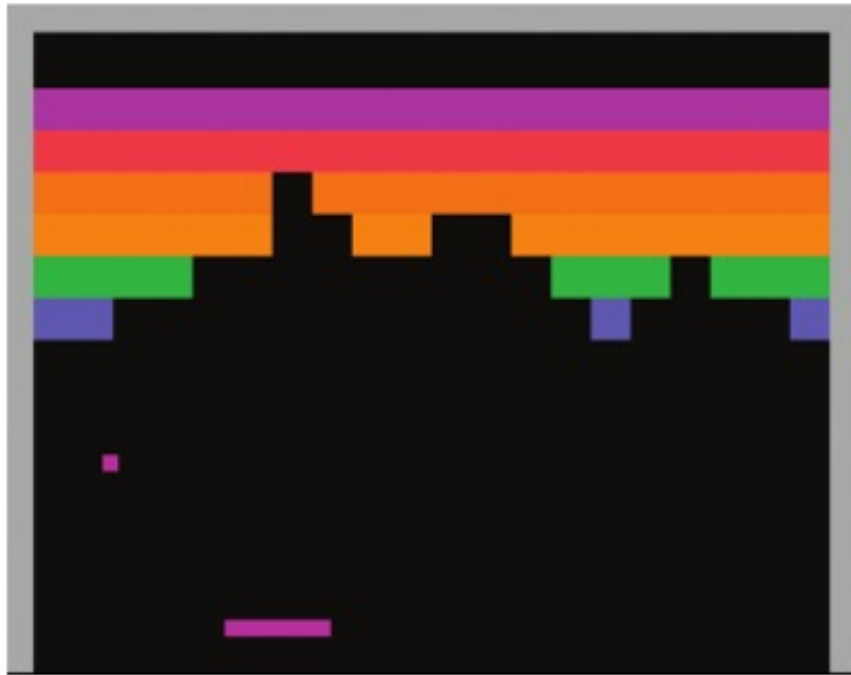


Game Evaluation Phase

Introduction

What are the limitations of Game Quality Assurance (QA)?

- ❖ Games have become much more complex → as computer hardware and software have improved
- ❖ **Limitation: Manual QA takes more time and effort**



Atari

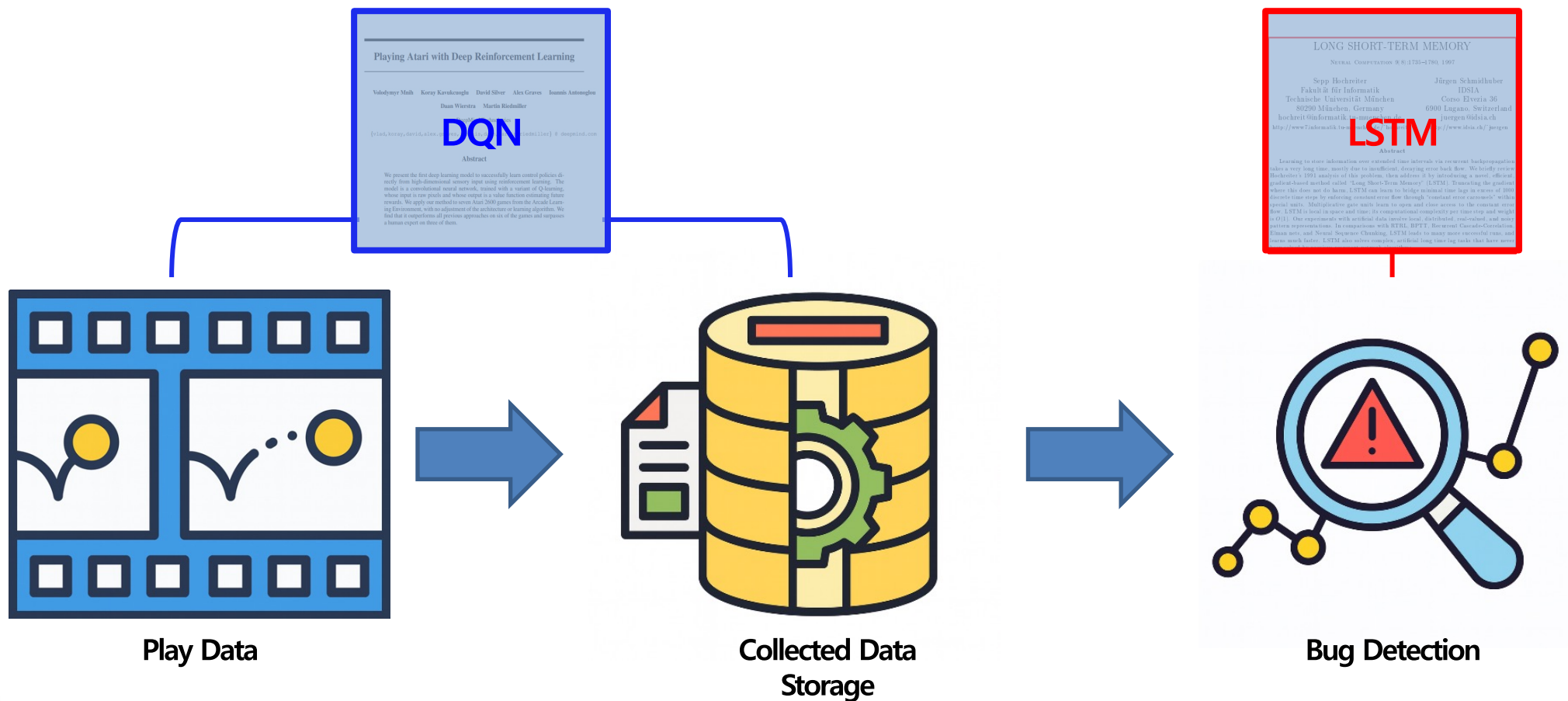


StarCraft2

Introduction

How Has AI Been Used for Game QA?

- ❖ To address this limitation, **researchers have started using AI to automate game QA**
- ❖ Well known study → AstroBug (2024, IEEE Transactions on Games)



Introduction

Why Is Multi-Agent Game Testing Needed?

- ❖ Previous AI-Based game testing studies focus on games with only one unit
- ❖ However, many recent games have **multiple units interacting at the same time**



StarCraft 2



League of Legends

Introduction

Why Is Multi-Agent Game Testing Needed?

- ❖ Previous AI-Based game testing studies focus on games with only one unit
- ❖ However, many recent games have **multiple units interacting at the same time**



StarCraft 2



League of Legends

Research Objective: Develop an Automated Game QA Framework for Multi-Unit Games

Preliminaries

What is Multi-Agent Reinforcement Learning (MARL)?

- ❖ A method where multiple agents interact with the environment and cooperate to learn an optimal action policy

Time step t



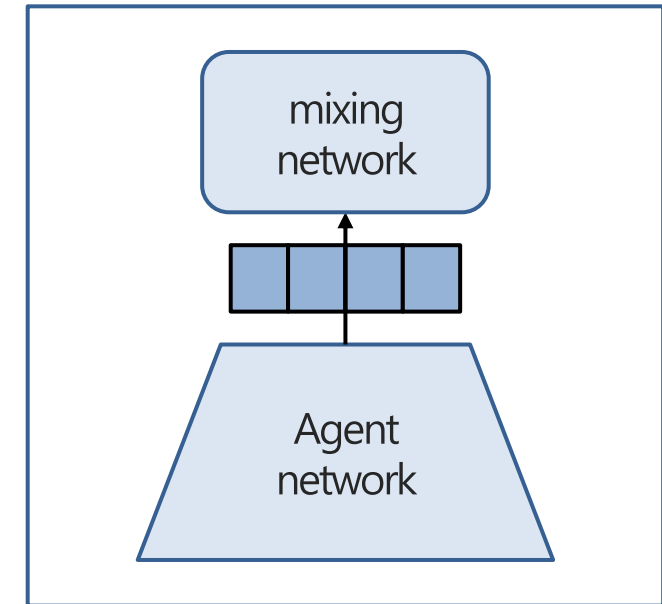
StarCraft II

Environment

① Provide the network with
information at time step t
(observations, available actions, etc.)



② Send the action selected
by the network at time step t



network (θ)

Agent

Preliminaries

What is Multi-Agent Reinforcement Learning (MARL)?

- ❖ Based on this information, the network decides **which action each agent should take and sends the selected actions**
- ❖ The environment changes based on these actions

Time step $t+1$



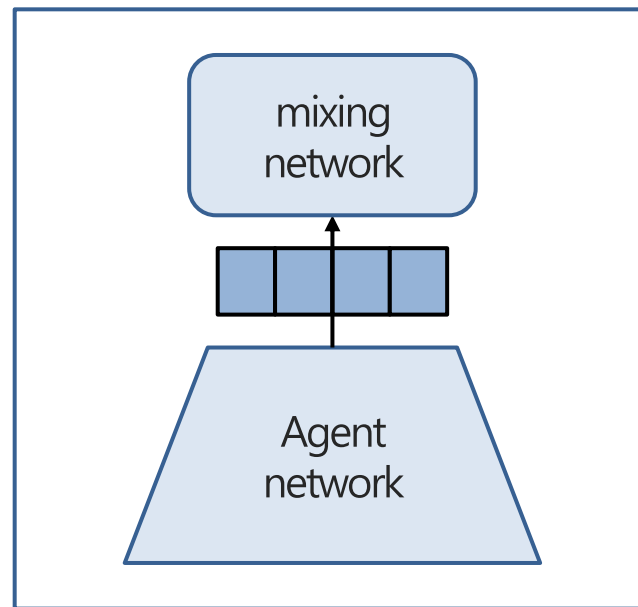
StarCraft II

Environment

① Provide the network with information at time step t (observations, available actions, etc.)



② Send the action selected by the network at time step t



network (θ)

Agent

Preliminaries

What is Multi-Agent Reinforcement Learning (MARL)?

- ❖ Episode: the data from the start to the end of one game

Time step $t+1$



StarCraft II

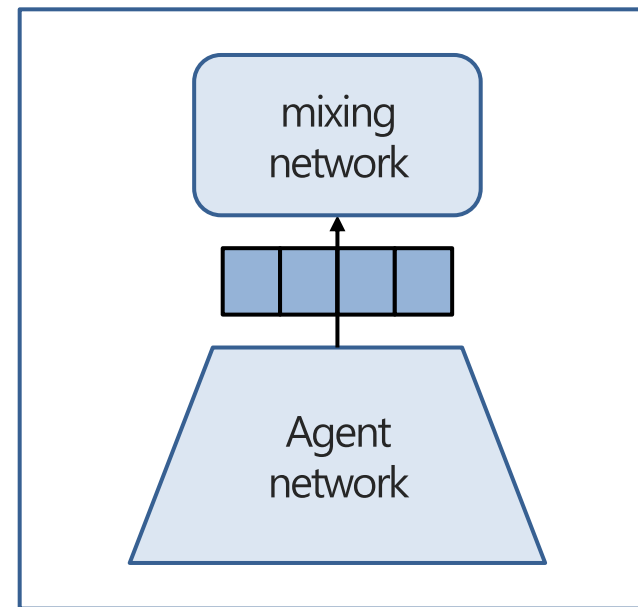
Environment

- ① Provide the network with information at time step t (observations, available actions, etc.)

Interact with the environment

Continue until the learning agents eliminate all enemies or are eliminated

- ② Send the action selected by the network at time step t



network (θ)

Agent

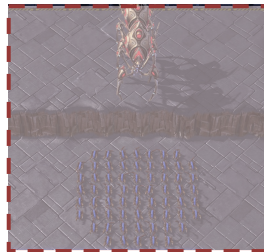
Preliminaries

What Is an Episode in MARL?

- ❖ Episode mainly contains 4 types of data: global state, observation, action, reward
- ❖ Global state: the entire game environment at each time step
- ❖ Observation: information that each agent can observe around it at each time step

Global State

	dimension						
Timestep	0	1.0	1.0	0.0	0.0	0.0	0.0
	1	0.78	0.69	0.0	0.0	0.0	0.0
	2	1.0	1.0	...	...	0.69	0.0
	...	...	...	...	...	...	...
	n	0.0	0.58	...	...	0.89	0.0



Observation

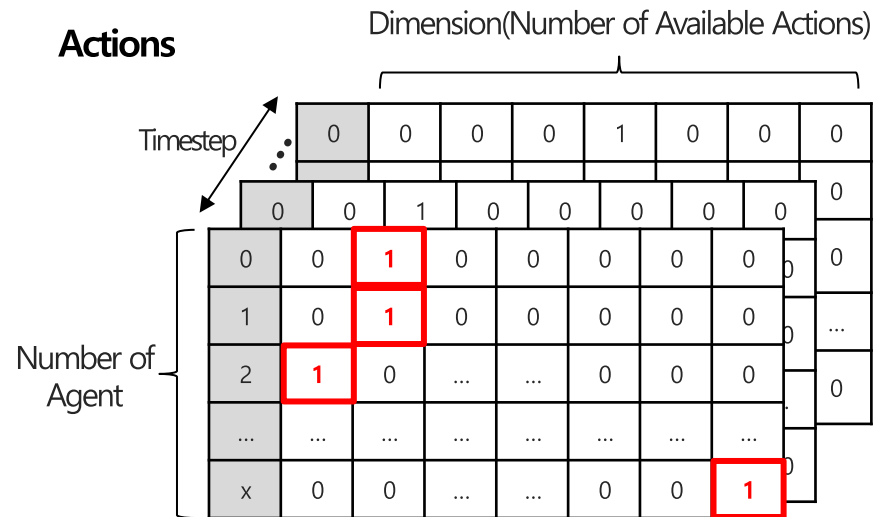
		dimension						
Number of Agent	Timestep	0	1.0	1.0	0.0	0.0	0.0	0.0
		0	1.0	1.0	0.0	0.0	0.0	0.0
	0	1.0	1.0	0.0	0.0	0.0	0.0	0.0
	1	0.78	0.69	0.0	0.0	0.0	0.0	0.0
	2	1.0	1.0	...	...	0.69	0.0	0.0
	x	0.0	0.58	...	...	0.89	0.0	0.0



Preliminaries

What Is an Episode in MARL?

- ❖ action is stored as a **one hot-vector**, where the selected action is **marked as one**
- ❖ Total reward is from the result of the agent's actions



Reward

Total Reward (Shared Reward)

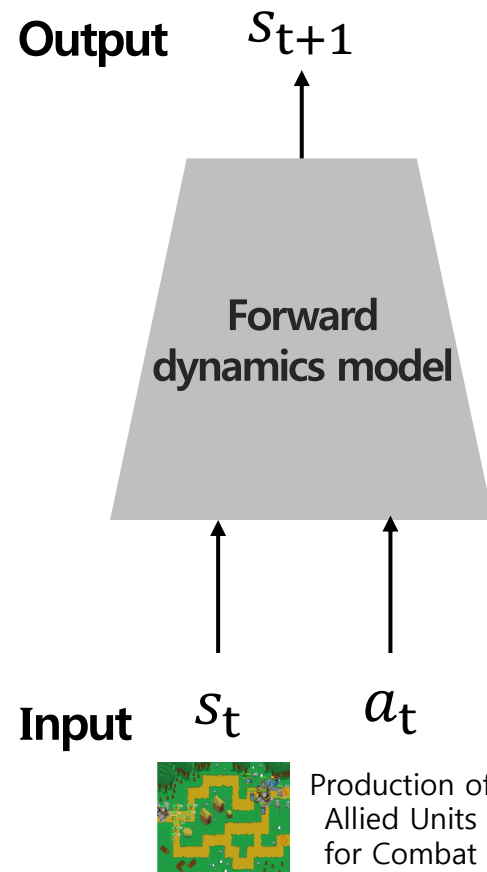
Timestep

0	0.23
1	0.72
2	0.78
...	...
n	0.0

Preliminaries

What Is a Forward Dynamics Model (FDM)?

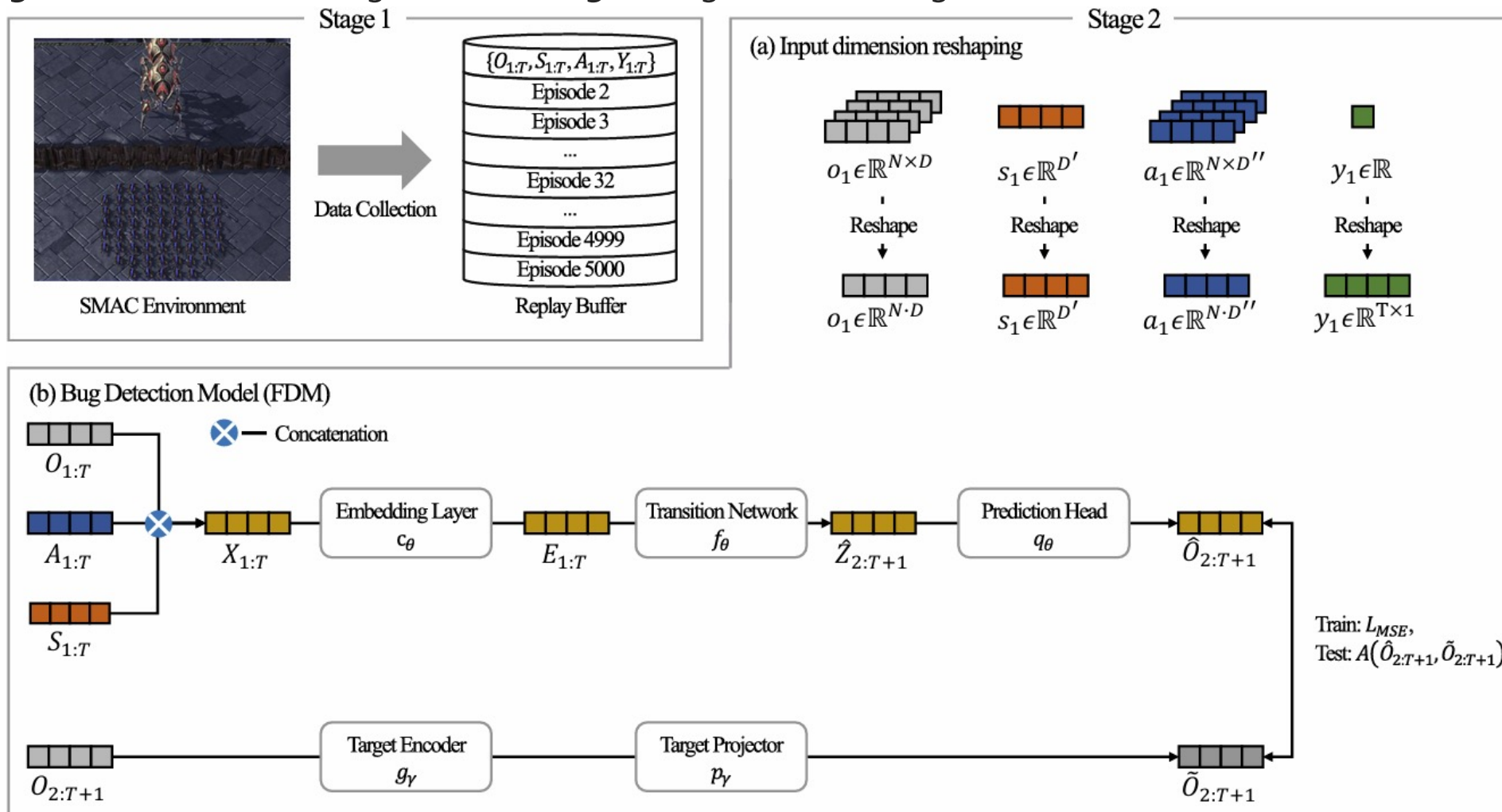
- ❖ Current state and action $\rightarrow$ the FDM predicts how the next state will change
- ❖ Using the FDM for bug detection $\rightarrow$ directly reflect these transition characteristics in the game



Methodology

How does Bughunter Work?

- ❖ **Stage 1:** Data Collection Using D-QMIX / **Stage 2:** Bug Detection Using FDM

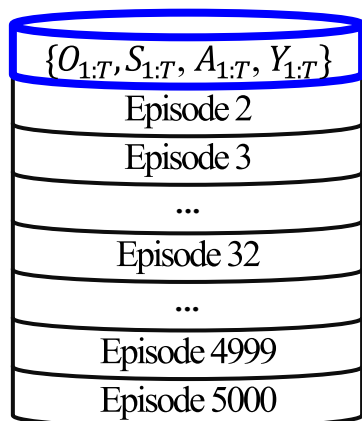


Methodology

How Do We Collect Gameplay data in stage 1?

- ❖ Using D-QMIX to interact with the environment for **1M time steps** and collect gameplay data in the replay buffer
- ❖ Inside the replay buffer, the data sequence from the start to the end of each game is stored as an episode

Replay Buffer



Episode

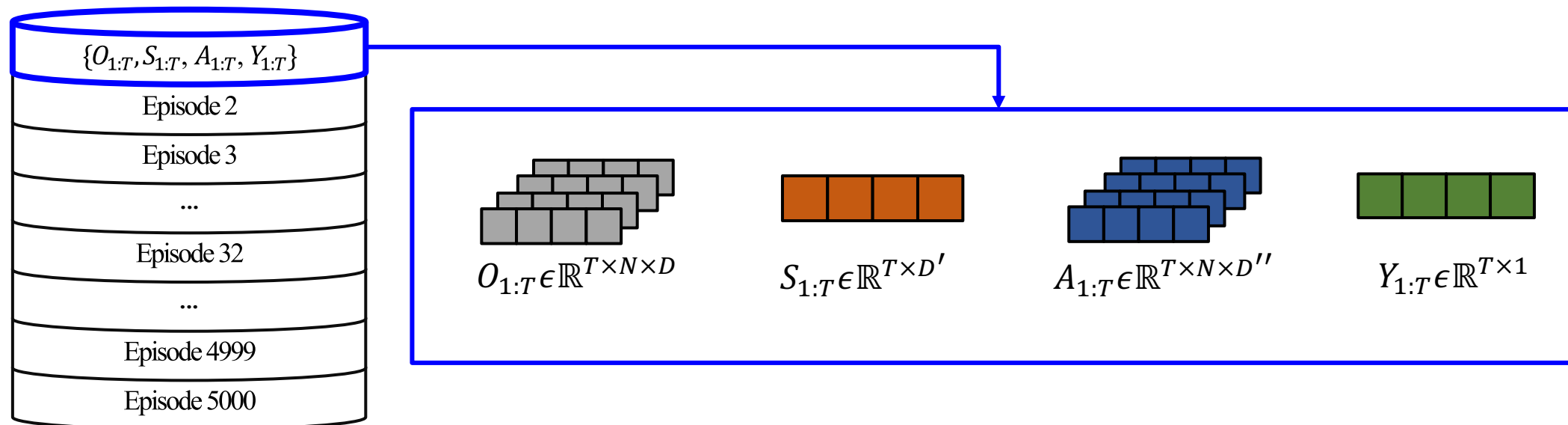


Methodology

What Data Do We Collect in stage 1?

- ❖ Each episode contains the **observation, global state, action and label data**
- ❖ Label shows whether each time step is normal or buggy

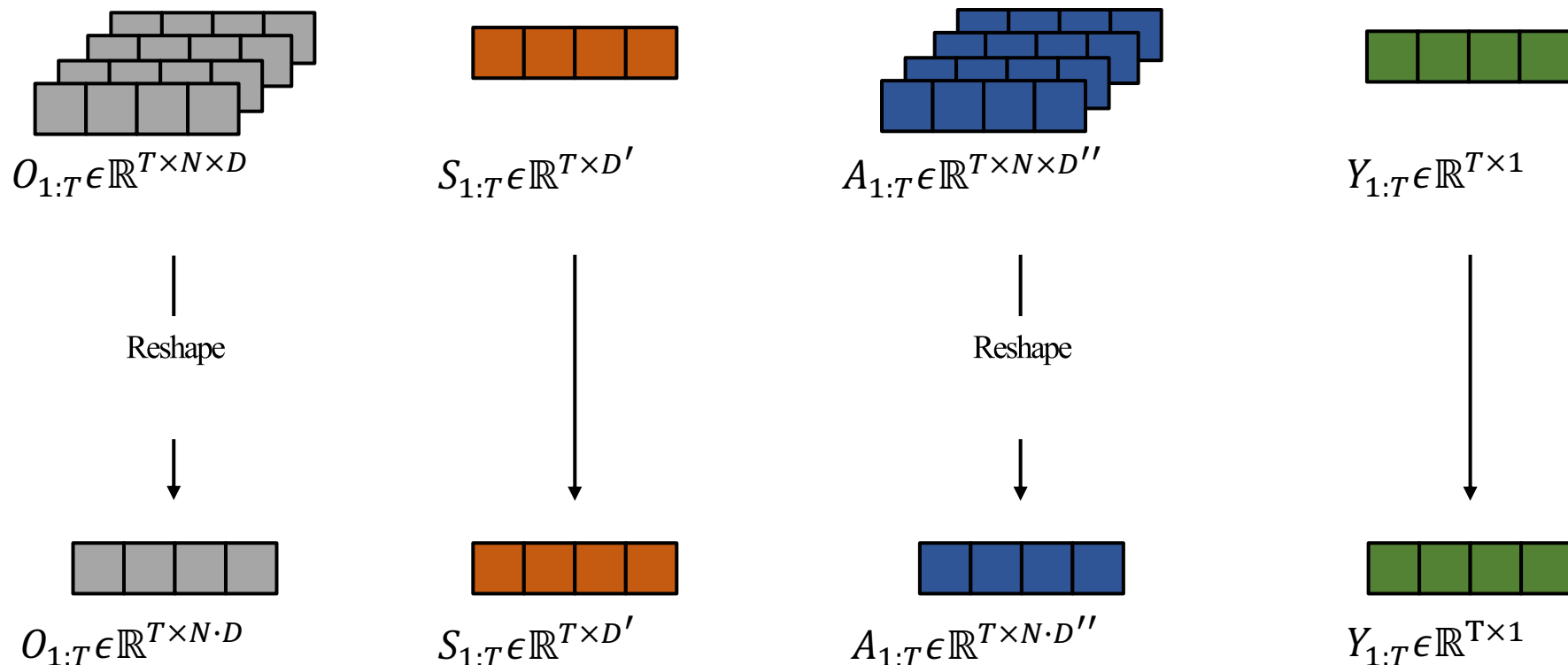
Replay Buffer



Methodology

How Do We Prepare the Data for Bug Detection?

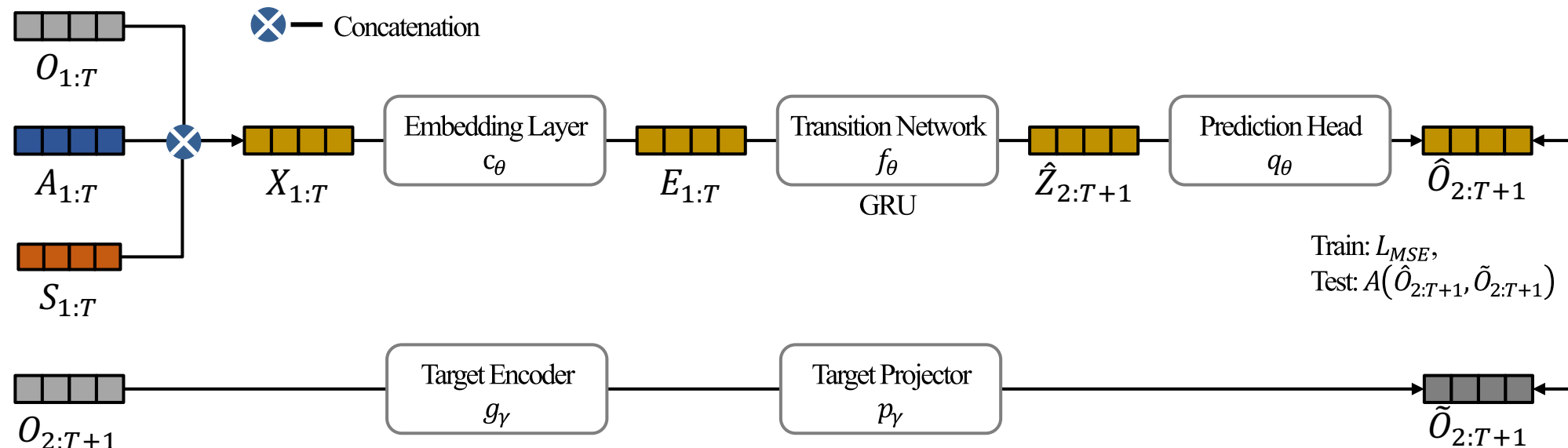
- ❖ Observation and action originally have 3 dimensions: **time step(T), agent(N), feature(D)**
- ❖ Reshape the data by flattening the information **from all agents into a single feature dimension**



Methodology

How Does FDM Detect Bugs?

- ❖ **Training:** Minimization of the mean squared error (MSE) loss
- ❖ **Evaluation:** MSE at each time step as the anomaly score
- ❖ When a bug breaks the normal transition pattern → prediction error becomes larger → detect the bug



Experiments

Where Do We Evaluate BugHunter?

- ❖ Experimental Environment: **SMAC**, Selected Scenarios: 2c_vs_64zg, 5m_vs_6m, and MMM2
- ❖ 2c_vs_64zg and 5m_vs_6m: 2 and 5 units of the same type / MMM2: a total of ten units from 3 different unit types



2c_vs_64zg



5m_vs_6m



MMM2

Experiments

What Types of Bugs Do We Consider?

- ❖ Definition of three bugs that can occur in games, **based on feedback from actual game developers**
 - Bug ①: A unit entering a specific area is treated as dead despite still being alive
 - Bug ②: An agent's health drops to 50% or below, its health is restored
 - Bug ③: Preventing a specific unit from attacking the enemy



Bug 1



Bug 2

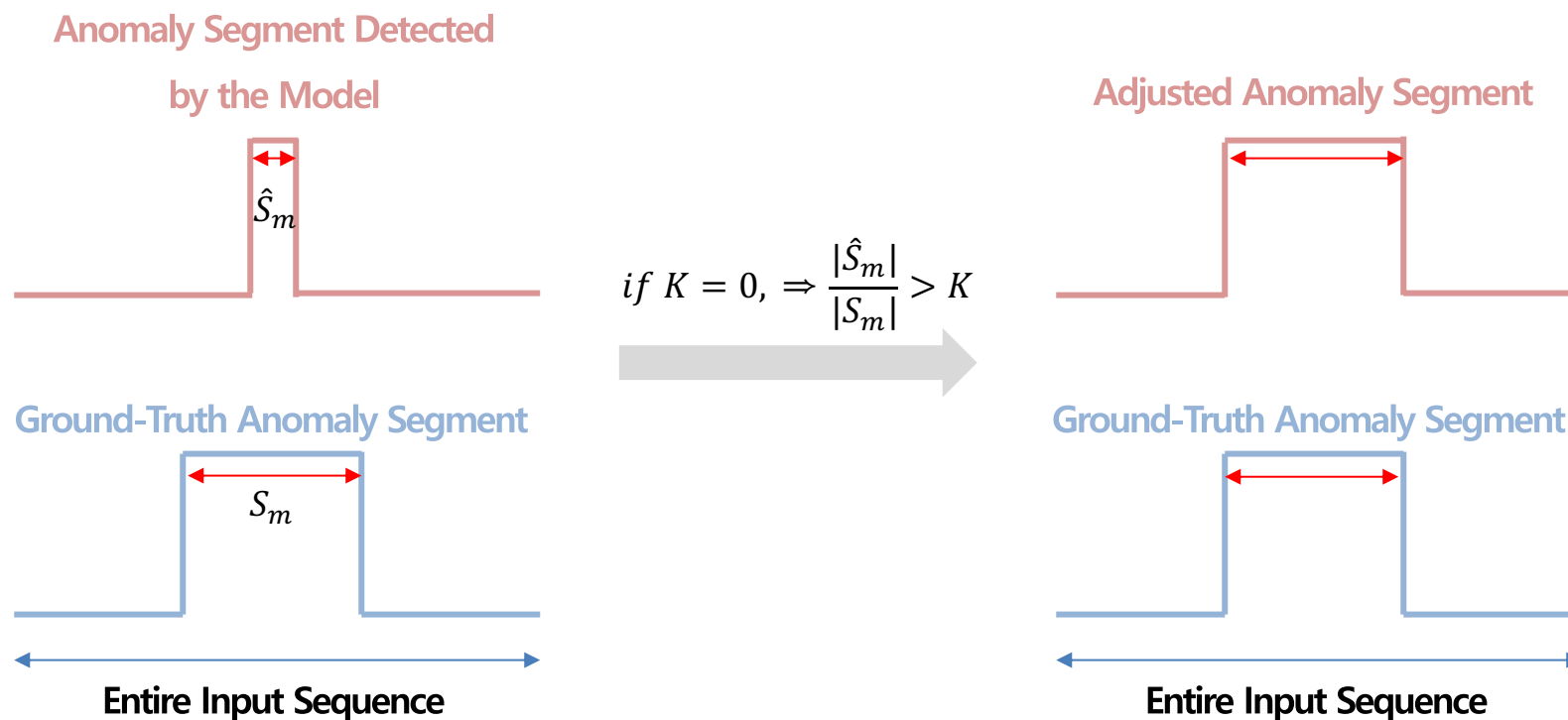


Bug 3

Experiments

What Does F1 at $K = 0$ Measure?

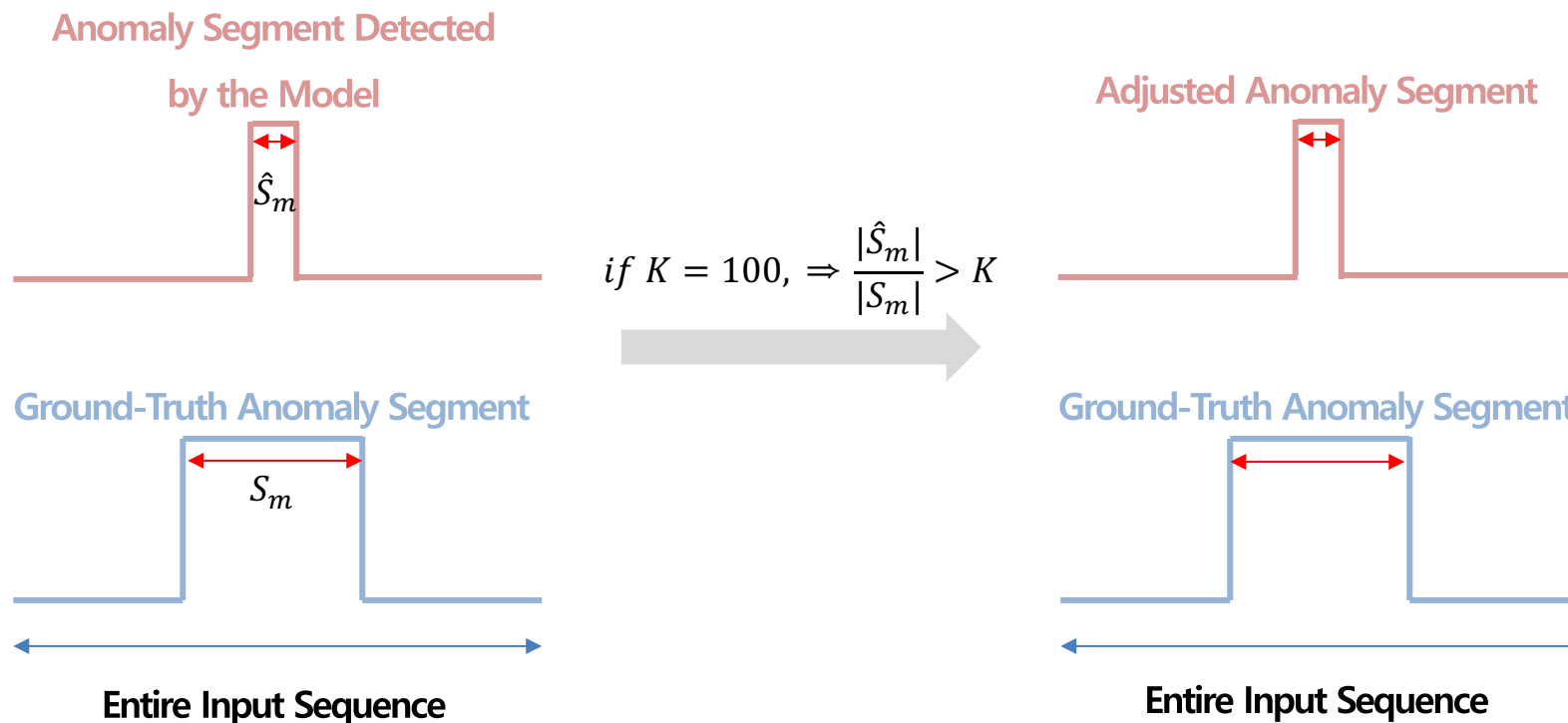
- ❖ Use of the **PA%K protocol** (AAAI, 2022), which is used in time-series anomaly detection research
- ❖ Time Steps Detected as Anomalous Within the Ground-Truth Anomaly Segment $\rightarrow \hat{S}_m = \{t' \in S_m | A(w_{t'}) > \delta\}$
- ❖ Ground-Truth Anomalous Time Steps Within the Anomaly Segment $\rightarrow S_m = \{t_{ms}, \dots, t_{me}\}$



Experiments

What Does F1 at $K = 100$ Measure?

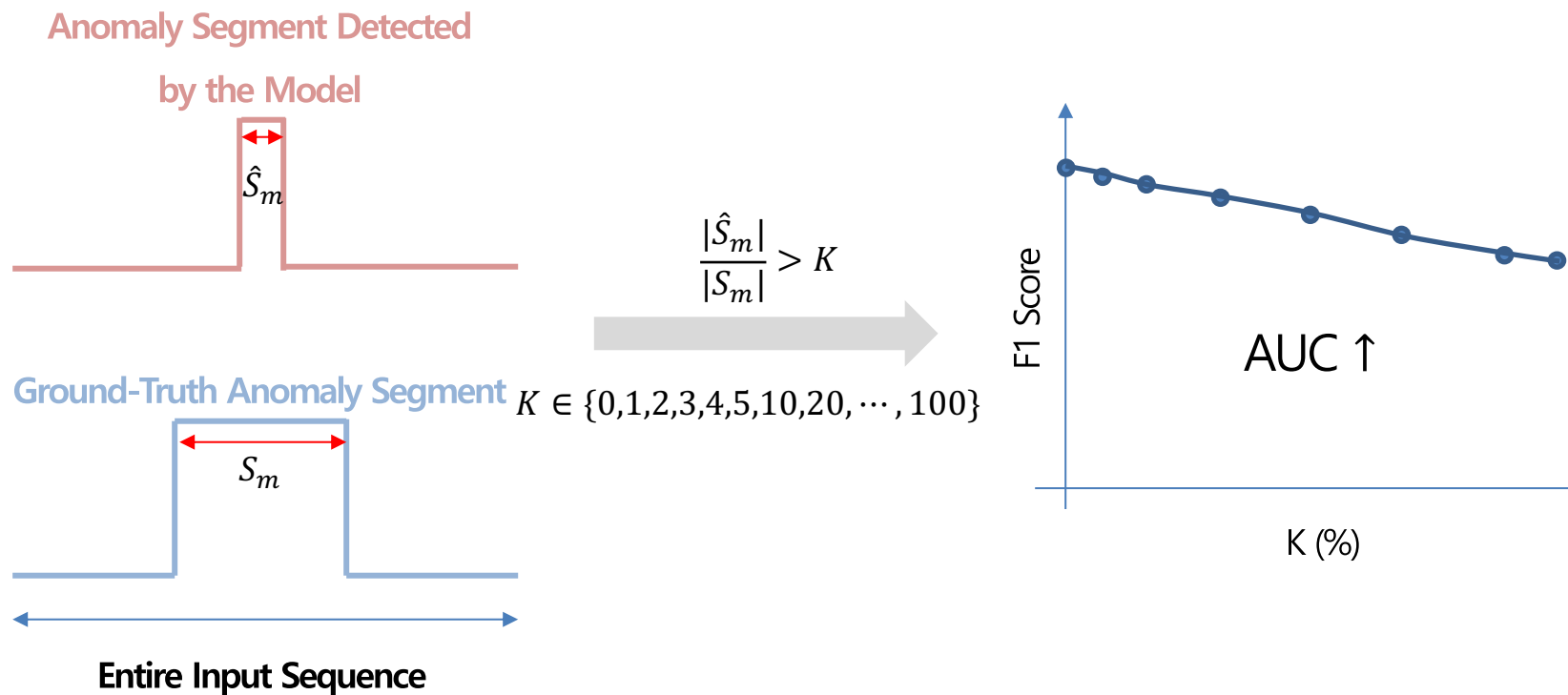
- ❖ $K = 100 \rightarrow$ it is the strictest condition
- ❖ At $k = 100$, **evaluating the model's actual detection results** without any additional adjustment



Experiments

What Does AUC?

- ❖ AUC: the area under the F1-K curve
- ❖ A higher AUC means the performance stays strong → **model can detect the whole bug segment more consistently**



Experiments

Results ① – Can FDM Outperform Recent Anomaly Detection Methods?

- ❖ Baseline: Anomaly Transformer(2022, ICLR), TranAD(2022, ACM), DCdetector(2023, ACM), Dual-TF(2024, ACM)
- ❖ Seven Repeated Experiments for Reliability and Consistency

Bold: Best Performance; Underlined: Second-Best Performance

Dataset	2c_vs_64zg								
Bug Type	1			2			3		
Metric	F1 (K=0)	F1 (K=100)	AUC	F1 (K=0)	F1 (K=100)	AUC	F1 (K=0)	F1 (K=100)	AUC
FDM (proposed)	74.84	20.95	32.39	65.89	33.15	44.59	60.48	38.29	46.17
Anomaly Transformer	<u>49.19</u>	4.74	8.06	<u>60.45</u>	<u>12.86</u>	<u>20.51</u>	51.62	12.45	18.72
DCdector	29.7	2.93	5.15	40.55	7.56	12.79	<u>54.41</u>	<u>16.57</u>	<u>24.02</u>
Dual-TF	0.0	0.0	0.0	4.01	3.93	3.95	5.91	5.89	5.9
TranAD	13.27	<u>10.15</u>	<u>11.61</u>	15.94	11.48	12.63	15.38	11.49	12.34

Experiments

Results ① – Can FDM Outperform Recent Anomaly Detection Methods?

- ❖ Baseline: Anomaly Transformer(2022, ICLR), TranAD(2022, ACM), DCdetector(2023, ACM), Dual-TF(2024, ACM)
- ❖ Seven Repeated Experiments for Reliability and Consistency

Bold: Best Performance; Underlined: Second-Best Performance

Dataset	5m_vs_6m								
Bug Type	1			2			3		
Metric	F1 (K=0)	F1 (K=100)	AUC	F1 (K=0)	F1 (K=100)	AUC	F1 (K=0)	F1 (K=100)	AUC
FDM (proposed)	62.27	49.18	55.91	65.25	55.34	63.2	47.38	47.02	47.25
Anomaly Transformer	<u>57.41</u>	12.73	<u>20.71</u>	92.83	<u>7.77</u>	13.22	<u>23.87</u>	10.52	14.43
DCdetector	35.95	10.07	15.72	<u>91.79</u>	6.4	11.72	9.13	4.72	6.17
Dual-TF	6.52	6.4	6.44	0.0	0.0	0.0	6.75	6.74	6.74
TranAD	23.48	<u>16.19</u>	19.19	66.36	7.62	<u>13.41</u>	21.32	<u>13.32</u>	<u>16.65</u>

Experiments

Results ① – Can FDM Outperform Recent Anomaly Detection Methods?

- ❖ Baseline: Anomaly Transformer(2022, ICLR), TranAD(2022, ACM), DCdetector(2023, ACM), Dual-TF(2024, ACM)
- ❖ Seven Repeated Experiments for Reliability and Consistency

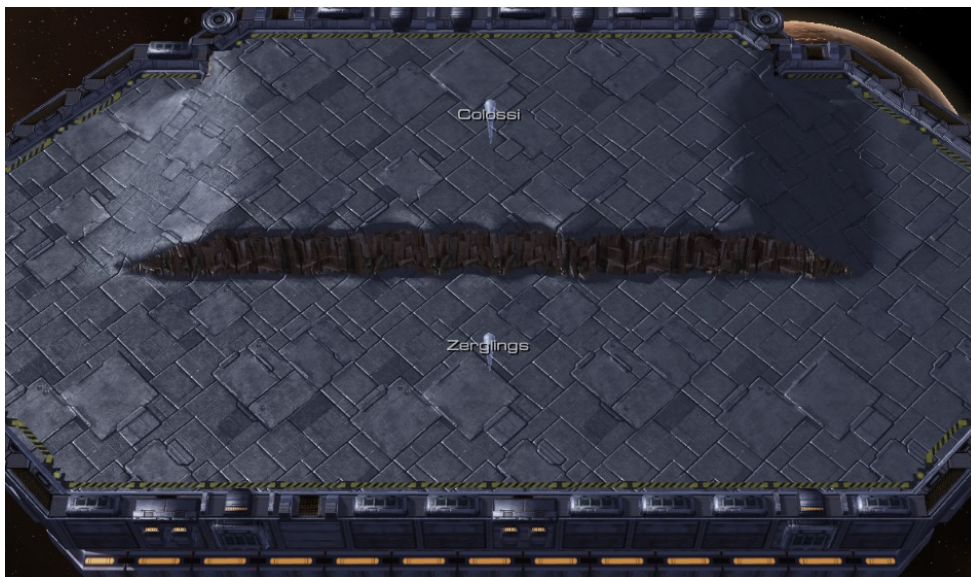
Bold: Best Performance; Underlined: Second-Best Performance

Dataset	MMM2								
Bug Type	1			2			3		
Metric	F1 (K=0)	F1 (K=100)	AUC	F1 (K=0)	F1 (K=100)	AUC	F1 (K=0)	F1 (K=100)	AUC
FDM (proposed)	<u>31.57</u>	26.76	29.8	87.39	63.84	76.17	47.12	44.22	45.72
Anomaly Transformer	6.79	0.82	1.53	<u>93.73</u>	4.58	7.73	2.22	1.25	1.54
DCdetector	61.38	<u>13.23</u>	<u>21.81</u>	94.41	<u>8.49</u>	<u>14.19</u>	<u>31.09</u>	<u>6.54</u>	<u>10.63</u>
Dual-TF	5.41	5.4	5.4	0.0	0.0	0.0	5.1	5.08	5.08
TranAD	4.23	0.57	1.01	51.36	2.56	4.86	6.9	2.69	3.7

Experiments

Results ② - Can a Pretrained FDM Generalize to Changed Game Environments without Retraining?

- ❖ In real games, the environment often changes **because of updates and map changes**
- ❖ Retraining the model every time the environment changes a little, is not practical



Original 2c_vs_64zg Scenario



Modified 2c_vs_64zg Scenario

Experiments

Results ② - Can a Pretrained FDM Generalize to Changed Game Environments without Retraining?

- ❖ The performance drops a little because the model never saw data from modified environment during training
- ❖ FDM achieved the highest AUC for all three bug types, **performed better overall than the baseline on F1 score**

Bold: Best Performance; Underlined: Second-Best Performance

Dataset	2c_vs_64zg (variation)					
Bug Type	1		2		3	
Metric	F1 (K=100)	AUC	F1 (K=100)	AUC	F1 (K=100)	AUC
FDM (proposed)	<u>12.2</u>	20.26	35.01	47.78	20.97	27.39
Anomaly Transformer	6.38	10.51	12.83	20.48	11.35	17.59
DCdetector	6.31	10.76	<u>27.07</u>	<u>39.38</u>	<u>17.66</u>	<u>25.34</u>
Dual-TF	0.0	0.0	5.24	5.25	4.72	4.72
TranAD	15.81	<u>16.98</u>	9.73	10.18	10.45	11.64

Conclusion

- ❖ **First automated bug detection framework** for multi-unit games
- ❖ **First application of an FDM** reflecting the characteristics of data collected through multi-agent reinforcement learning to game bug detection
- ❖ Construction of **a new benchmark** for future research on bug detection in multi-unit games
- ❖ **Superior performance of the FDM** over recent time-series anomaly detection methods and verification of its **strong generalization capability**

Q & A

Thank you

Appendix

Numbers of Collected Training and Evaluation Samples and Normal-to-Bug Ratios

- ❖ Generation of normal and bug data by injecting three types of bugs into each scenario
- ❖ Adjustment of the normal-to-anomalous data ratios in the evaluation set to 7:3, 8:2, and 9:1 based on the collected data

Scenario		2c_vs_64zg	5m_vs_6m	MMM2
Train		100,000	62,500	100,000
Test	Bug Type 1	141,240 (9:1)	28,900 (8:2)	18,560 (8:2)
	Bug Type 2	196,100 (8:2)	62,500 (9:1)	100,000 (9:1)
	Bug Type 3	16,840 (7:3)	25,000 (7:3)	18,720 (7:3)

Appendix

Results ① – Can FDM Outperform Recent Anomaly Detection Methods?

❖ AUC Results for Bug Type 2 in the MMM2 Scenario

